

CLAUDE CODE FIELD GUIDE · KOREAN EDITION

Claude Code

실전 완전정복

설정팩과 함께 배우는 AI 코딩 에이전트 실무 활용 가이드
CLAUDE.md · 커스텀 커맨드 · 서브에이전트 · 훅 · MCP

커맨드 35종

에이전트 11종

훅 15종 동봉

```
$ npm install -g @anthropic-ai/claude-code
```

목차

프롤로그 — 이 책의 사용법

1장. 설치와 첫 실행 — 5분 안에 끝내기

2장. CLAUDE.md — 프로젝트의 기억을 설계한다

3장. settings.json — 권한과 환경을 통제한다

4장. 커스텀 슬래시 커맨드 — 반복 작업을 명령어로 만든다

5장. 서브에이전트 — 역할별 전문가를 고용한다

6장. 훅(Hooks) — 사람이 안 지켜봐도 되는 가드레일

7장. MCP 서버 — Claude의 손을 외부 세계로 확장한다

8장. 실전 워크플로우 4종 — 조합이 실력이다

9장. 컨텍스트와 비용 관리 — 오래 달리기 위한 기술

10장. 트러블슈팅 FAQ — 자주 부딪히는 문제들

부록. 동봉 설정팩(claude-forge) 설치 가이드

맺으며 — 도구가 아니라 시스템을 만드세요

(지원되는 PDF 뷰어에서는 목차 항목 클릭으로 이동할 수 있습니다)

프롤로그 — 이 책의 사용법

Claude Code를 "설치해서 채팅하듯 쓰는 것"과 "제대로 세팅해서 쓰는 것"의 생산성 차이는 체감상 3~5배 이상입니다. 하지만 공식 문서는 영어이고, 한국어 자료 대부분은 설치법 수준에서 멈춰 있습니다. CLAUDE.md 설계, 커스텀 커맨드, 서브에이전트, 훅(Hooks), MCP 연동까지 다루는 한국어 실전 자료는 아직 드뭅니다.

이 책의 3가지 약속

- 모든 예제는 **실제로 작동하는 설정 파일**입니다. 동봉 설정팩(claude-forge)에는 커스텀 커맨드 35종, 서브에이전트 11종, 자동화 훅 15종, 규칙 파일 8종이 들어 있고, 책의 예제는 전부 여기서 발췌했습니다.
- 모든 장은 **하나의 실전 프로젝트 사례로 연결**됩니다. 개념 설명으로 끝나지 않고, 실제 팀이 그 기법을 적용하는 터미널 장면과 결과까지 보여드립니다.
- "이런 게 가능하다" 수준이 아니라 **파일 경로, JSON 키, 스크립트 내용**까지 그대로 따라 할 수 있게 씁니다.

관통 사례 소개 — 리프오더 팀

이 책 전체에서 함께할 사례 팀입니다. 카페 대상 원두 정기배송 주문 관리 SaaS "리프오더"를 만드는 2인 개발팀 — 백엔드의 지우, 프론트엔드의 하람. 두 사람 다 Claude Code를 쓰고는 있었지만, 세팅 없이 "채팅하듯" 쓰다가 다음 문제를 겪고 있었습니다.

- 세션마다 프로젝트 규칙을 다시 설명하느라 첫 10분을 소모
- Claude가 만든 코드가 팀 컨벤션과 달라 리뷰에서 반려되는 일이 반복
- 위험한 명령(DB 초기화, 강제 푸시)을 실행할 뻔한 아찔한 순간 2회
- 매 승인 버튼 클릭에 지쳐 자동 승인 모드를 켜다가 더 불안해짐

각 장의 마지막 **[현장 적용]** 섹션에서, 이 문제들이 하나씩 시스템으로 해결되는 과정을 따라갑니다.

대상 독자: Claude Code를 써봤지만 "좋은 채팅" 수준으로 쓰고 있는 개발자, 바이브코딩으로 시작했는데 프로젝트가 커지며 한계를 느끼는 분, 팀에 AI 코딩 도구를 도입하려는 리더.

1장. 설치와 첫 실행 — 5분 안에 끝내기



Claude Code는 터미널에서 동작하는 AI 코딩 에이전트입니다. 채팅창에 코드를 붙여넣는 방식과 달리, 에이전트가 직접 파일을 읽고, 수정하고, 테스트를 돌리고, 커밋합니다.

실행 절차

1. **설치**: 터미널에서 한 줄이면 됩니다.

```
npm install -g @anthropic-ai/claude-code
```

macOS는 Homebrew(`brew install claude-code`), Windows는 WSL 환경을 권장합니다. 네이티브 Windows도 지원되지만 심볼릭 링크 기반 설정 공유가 제한됩니다.

2. **로그인**: 프로젝트 폴더에서 `claude` 를 실행하면 브라우저 인증이 열립니다. Claude Pro/Max 구독 또는 API 키 어느 쪽으로도 사용할 수 있습니다.

3. **첫 실행 확인**: 아무 프로젝트 폴더에서 구조 파악부터 시켜보세요.

4. **프로젝트 초기화**: `/init` 을 실행하면 Claude가 코드베이스를 분석해 CLAUDE.md 초안을 만들어줍니다. (2장에서 자세히)

알아두면 좋은 실행 모드

모드	명령	용도
대화형	<code>claude</code>	기본. 터미널에서 대화하며 작업
헤드리스	<code>claude -p "작업 내용"</code>	스크립트/CI에서 일회성 실행
계속하기	<code>claude -c</code>	직전 세션 이어서
플랜 모드	세션 중 Shift+Tab	수정 없이 계획만 세우게 하기

팁: 처음 며칠은 플랜 모드를 적극 쓰세요. Claude가 뭘 하려는지 계획을 먼저 보고

승인하는 습관이 들면, 나중에 자율 실행을 시켜도 결과를 예측할 수 있게 됩니다.

CASE STUDY 현장 적용 — 리프오더 저장소에서의 첫 세션

지우가 리프오더 백엔드 저장소에서 처음 Claude Code를 실행한 장면입니다.

```
claude - 세션 기록

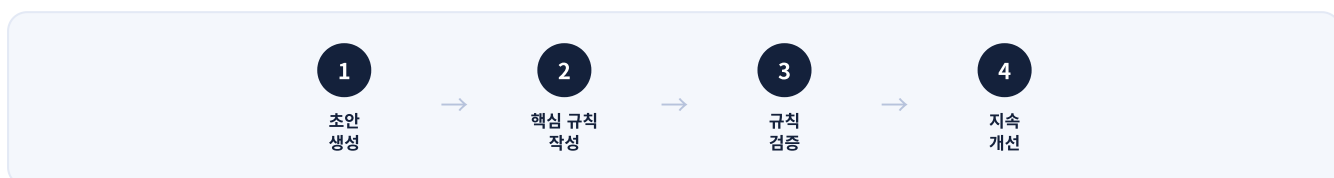
$ cd leaforder-api && claude

* Claude Code v2.x - leaforder-api

> 이 프로젝트 구조를 파악하고, 테스트 실행 방법을 알려줘
• 프로젝트를 분석하겠습니다.
• Glob(pattern: "**/*.ts")
  | 147개 파일
• Read(package.json)
  | 34 lines
• 분석 결과:
  - Express 4 + TypeScript, Prisma(PostgreSQL), vitest
  - 도메인: orders/, subscriptions/, roasters/
  - 테스트: npm test (vitest run)
  - 주의: src/legacy/ 폴더는 마이그레이션 중인 구버전 코드
> /init
• CLAUDE.md 초안을 생성했습니다. 검토 후 수정해주세요.
```

첫 세션에서 지우가 배운 것: Claude는 프로젝트를 "추측"하지 않고 직접 파일을 읽어 파악하며, 그 파악 결과를 CLAUDE.md로 고정해두면 다음 세션부터는 이 10분이 0분이 됩니다. 그 방법이 바로 다음 장입니다.

2장. CLAUDE.md — 프로젝트의 기억을 설계한다



Claude Code는 세션을 시작할 때 프로젝트 루트의 `CLAUDE.md`를 자동으로 읽습니다. 여기 적힌 내용은 매 대화에 항상 적용되는 "프로젝트 헌법"이 됩니다. 이 파일의 품질이 곧 결과물의 품질입니다.

실행 절차

1. **생성:** `/init`으로 초안을 만들거나 직접 작성합니다.
2. **핵심 규칙 작성:** 아래 골격은 반드시 채우세요.

```
# 프로젝트: 리프오더 주문 관리 API

## 기술 스택
- TypeScript 5 + Node.js 22, Express, Prisma, PostgreSQL

## 명령어
- 테스트: npm test / 린트: npm run lint / 빌드: npm run build

## 코딩 규칙
- 모든 금액 계산은 정수(원 단위). 부동소수점 금지
- API 응답은 반드시 lib/responses.ts 헬퍼로 감쌀 것
- DB 마이그레이션 파일은 직접 수정 금지

## 하지 말 것
- .env 파일 읽거나 수정하지 말 것
- src/legacy/ 코드를 참조 패턴으로 삼지 말 것
- main 브랜치에 직접 커밋하지 말 것
```

3. **검증**: 새 세션에서 규칙에 어긋나는 요청을 일부러 해보세요. Claude가 규칙을 근거로 거절하거나 대안을 제시하면 성공입니다.
4. **지속 개선**: Claude가 같은 실수를 반복하면 그 자리에서 `#` 키로 규칙을 추가하세요. CLAUDE.md는 실수가 누적될수록 강해지는 문서입니다.

잘 쓴 CLAUDE.md의 3원칙

- **짧게**: 모든 세션의 컨텍스트를 차지합니다. 장황한 설명보다 명령형 규칙 위주로.
- **구체적으로**: "코드를 깔끔하게"(나뭇) → "함수 30줄 이내, 매개변수 4개 이하"(좋음)
- **금지 목록 포함**: 해야 할 것보다 하지 말아야 할 것이 사고를 더 잘 막습니다.

CASE STUDY 현장 적용 — "왜 자꾸 legacy 코드를 베끼지?"

리프오더 팀의 고질병: Claude가 새 API를 만들 때 자꾸 `src/legacy/` 의 낡은 패턴 (콜백 기반, 응답 헬퍼 미사용) 을 참조하는 것이었습니다. CLAUDE.md에 금지 규칙을 추가하기 전과 후의 차이입니다.

```
claude - 세션 기록

— CLAUDE.md 규칙 추가 전 —
> 로스터리 검색 API 추가해줘
• src/legacy/search.ts 패턴을 참고해 구현하겠습니다...
  (콜백 기반 + 응답 헬퍼 미사용 코드 생성 → 리뷰 반려)

— CLAUDE.md에 "src/legacy/ 참조 금지" 추가 후 —
> 로스터리 검색 API 추가해줘
• CLAUDE.md 규칙에 따라 src/legacy/는 참조하지 않고,
  orders/ 도메인의 현행 패턴(async/await + responses.ok)을
  기준으로 구현하겠습니다.
• Write(src/roasters/search.ts)
  | 58 lines - 컨벤션 준수
```

하람의 한 줄 회고: "규칙 한 줄 추가가 리뷰 반려 왕복 이틀을 없앴다." 이 팀은 이후 2주간 반려 사유가 나올 때마다 규칙을 한 줄씩 승격했고, CLAUDE.md는 23줄짜리 짧은 문서를 유지하면서도 리뷰 반려율을 눈에 띄게 줄였습니다.

리뷰 반려(컨벤션 사유)

주 4~5건 → 주 0~1건

규칙 승격 2주 후

세션 시작 설명 시간

약 10분 → 0분

CLAUDE.md 자동 로드

CLAUDE.md 분량

23줄

짧게 유지하는 게 핵심

3장. settings.json — 권한과 환경을 통제한다

`.claude/settings.json` (프로젝트) 또는 `~/.claude/settings.json` (전역)으로 Claude Code의 동작을 세밀하게 제어합니다. 특히 **권한(permissions)** 설정은 안전과 생산성을 동시에 잡는 핵심입니다.

자주 쓰는 설정 예시

```
{
  "permissions": {
    "allow": [
      "Bash(npm test:*)",
      "Bash(npm run lint:*)",
      "Bash(git status)",
      "Bash(git diff:*)"
    ],
    "deny": [
      "Bash(rm -rf:*)",
      "Read(.env*)",
      "Read(**/secrets/**)"
    ]
  }
}
```

테스트/린트/조회성 git 명령은 승인 없이 자동 실행되고, 위험한 명령과 시크릿 파일 접근은 아예 차단됩니다.

권한 설계의 사고방식

1. 처음엔 기본값(매번 확인)으로 시작합니다.
2. 반복해서 승인하게 되는 안전한 명령을 `allow`에 추가합니다.
3. 절대 일어나면 안 되는 일을 `deny`에 추가합니다. `deny`가 `allow`보다 우선합니다.
4. 팀 프로젝트라면 `.claude/settings.json`을 git에 커밋해 팀 전체가 같은 가드레일을 공유하고, 개인 취향은 `.claude/settings.local.json`(git 제외)에.

경고: `--dangerously-skip-permissions` 플래그는 모든 확인을 건너뛸니다. 격리된

컨테이너가 아니라면 쓰지 마세요. 편하다고 로컬에서 켜두는 순간 "rm -rf 사고"의

주인공이 될 수 있습니다.

CASE STUDY 현장 적용 — 승인 피로에서 팀 표준 가드레일로

지우는 하루에 승인 버튼을 60번 넘게 누르고 있었습니다. 그렇다고 전체 자동 승인은 불안했죠. 해결은 "자주 누르는 것만 골라서 허용"이었습니다.

```
claude - 세션 기록

> /permissions
• 최근 세션에서 반복 승인된 명령 상위:
  1. Bash(npm test)          31회
  2. Bash(npm run lint)      18회
  3. Bash(git diff ...)      14회
  4. Bash(npx prisma studio)  2회
  → 1~3번을 allow 목록에 추가할까요? (prisma studio는
    DB를 여는 명령이라 수동 승인 유지를 권장합니다)
> 응, 1~3번만 추가하고 settings.json은 git에 커밋해줘
• Edit(.claude/settings.json) - allow 3건 추가
• Bash(git add .claude/settings.json && git commit ...)
  └─ [main a3f12c] chore: 팀 공용 권한 설정 추가
```

이 커밋 하나로 하람의 환경에도 같은 가드레일이 적용됐습니다. 팀 온보딩 문서에 "권한 설정법"을 쓸 필요가 없어진 것이 부수 효과.

일일 승인 클릭

60여 회 → 10회 내외

allow 3줄의 효과

위험 명령 차단

deny 4패턴

rm -rf, .env, secrets, force push

팀 공유

git 커밋 1회

온보딩 문서 불필요

4장. 커스텀 슬래시 커맨드 — 반복 작업을 명령어로 만든다



`.claude/commands/` 폴더에 마크다운 파일을 넣으면 파일명이 곧 슬래시 커맨드가 됩니다. "매번 길게 설명하던 반복 요청"을 한 단어로 압축하는 기능입니다.

실행 절차

1. **파일 생성:** `.claude/commands/quick-commit.md`

2. **프롬프트 작성:** 파일 내용이 곧 실행될 지시문입니다.

```
---
description: 변경사항을 분석해 컨벤션에 맞는 커밋 메시지로 커밋
allowed-tools: Bash(git:*), Read
---
```

현재 스테이징된 변경사항을 분석하고:

1. `conventional commits` 형식(`feat/fix/refactor/docs`)으로 커밋 메시지를 작성하라
2. 제목은 50자 이내, 본문에는 "왜"를 적어라
3. 커밋을 실행하라

3. **인자 처리:** 지시문 안에 `$ARGUMENTS` 를 쓰면 `/quick-commit 긴급 수정` 처럼 호출 시 넘긴 인자가 치환됩니다.

4. **테스트:** 새 세션에서 실행해 의도대로 동작하는지 확인합니다.

설정팩 커맨드 중 활용도 높은 것들

커맨드	하는 일
<code>/plan</code>	구현 전 계획 수립을 강제. 큰 작업 전 필수
<code>/tdd</code>	테스트 먼저(RED) → 최소 구현(GREEN) → 리팩토링 사이클 강제
<code>/code-review</code>	심각도(CRITICAL~LOW) 등급을 매기는 체계적 코드 리뷰
<code>/security-review</code>	OWASP 기준 보안 점검
<code>/verify-loop</code>	빌드·테스트·린트가 모두 통과할 때까지 자동 반복 수정
<code>/checkpoint</code>	긴 작업 중간 저장
<code>/worktree-start</code>	git worktree로 병렬 작업 브랜치 생성

핵심 통찰: 좋은 커맨드는 "요청"이 아니라 "절차"를 담습니다. "코드 리뷰해줘"가 아니라 "1단계: 스펙 준수 확인 → 2단계: 보안 점검 → 3단계: 심각도 등급 → 4단계: 파일:줄번호와 수정안 제시"처럼 절차를 명시할수록 결과가 일관됩니다.

CASE STUDY 현장 적용 — 커밋 메시지 시비가 사라진 날

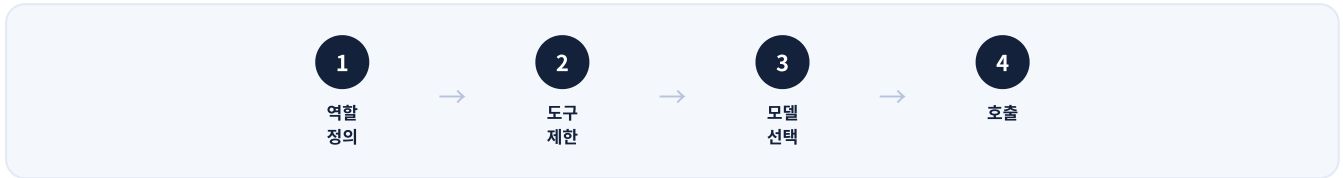
리프오더 팀의 커밋 로그는 "fix", "수정", "asdf"로 얼룩져 있었습니다. `/quick-commit` 도입 후의 장면입니다.

```
claude - 세션 기록

> /quick-commit
• Bash(git diff --staged --stat)
  | src/orders/refund.ts | 41 ++++++
  | src/orders/refund.test.ts | 66 ++++++++
• 변경 분석: 부분 환불 시 배송비 처리 로직 추가 + 테스트
• Bash(git commit -m "feat(orders): 부분 환불 시 배송비 정산 로직 추가
  전액 환불과 달리 부분 환불은 배송비를 차감하지 않아야 함.
  QA에서 발견된 이슈 #142의 근본 원인 수정.")
  | [main 7be03d1]
```

커밋 메시지에 "왜"가 남기 시작하자, 3개월 뒤 히스토리를 뒤지던 하람이 이슈의 맥락을 커밋 본문만으로 복원할 수 있었습니다. 커맨드 파일 12줄이 만든 변화입니다.

5장. 서브에이전트 — 역할별 전문가를 고용한다



`.claude/agents/` 폴더의 마크다운 파일은 각각 독립된 컨텍스트에서 동작하는 서브에이전트가 됩니다. 메인 세션의 컨텍스트를 오염시키지 않고 전문 작업을 위임할 수 있습니다.

실행 절차

1. **역할 정의**: 에이전트 하나 = 역할 하나. "다 잘하는 에이전트"는 아무것도 잘하지 못합니다.
2. **도구 제한**: 리뷰어에게 Write 권한을 주지 마세요. 읽기 전용 에이전트는 사고를 칠 수 없습니다.
3. **모델 선택**: 깊은 추론이 필요한 역할(아키텍트, 리뷰어)은 상위 모델, 기계적 실행(빌드 에러 수정, 문서 갱신)은 빠른 모델로.
4. **호출**: 이름을 불러 명시적으로, 또는 description 조건으로 자동 위임.

실제 에이전트 정의 (설정팩 `agents/code-reviewer.md` 발췌)

```

---
name: code-reviewer
description: 코드 품질·보안 전문 리뷰어. 코드 수정 후 반드시 사용.
tools: ["Read", "Grep", "Glob", "Bash"]
model: opus
---

```

너는 코드 리뷰어다. 임무: 체계적이고 심각도 기반의 리뷰.

성공 기준:

- 스펙 준수를 코드 품질보다 먼저 검증한다
- 모든 지적은 파일:줄번호를 명시한다
- 심각도 등급: CRITICAL / HIGH / MEDIUM / LOW
- 각 지적마다 구체적 수정안을 제시한다
- 결론은 셋 중 하나: APPROVE / REQUEST CHANGES / COMMENT

주목: `tools`에 Write가 없습니다. 이 리뷰어는 지적만 할 수 있고 고치지는 못합니다. "리뷰하다가 멋대로 코드를 바꿔버리는" 사고를 구조적으로 차단합니다.

설정팩의 에이전트 구성 (11종)

- 설계·계획: `architect`, `planner`
- 품질: `code-reviewer`, `security-reviewer`, `database-reviewer`, `tdd-guide`
- 실행·수리: `build-error-resolver`, `refactor-cleaner`, `e2e-runner`
- 유지보수: `doc-updater`, `verify-agent`

CASE STUDY 현장 적용 — 배포 전 15분, 리뷰어 에이전트의 CRITICAL

금요일 배포 직전, 지우가 습관처럼 리뷰어를 돌린 장면입니다.

```

claude - 세션 기록

> code-reviewer로 이번 브랜치 변경분 검토해줘
• Task(code-reviewer)
  └─ 검토 대상: 7 files, +412 -87
    ── 리뷰 리포트 ───────────────────
    [CRITICAL] src/webhooks/pay.ts:34
      서명 검증 없이 결제 웹훅 body를 신뢰함.
      위조 요청으로 주문 상태 변조 가능.
      수정안: verifySignature(req) 실행 후 처리.
    [MEDIUM] src/orders/list.ts:71
      페이지네이션 누락 — 주문 1만 건 이상 시 응답 지연.
    [LOW] 네이밍 2건 (상세 생략)
    결론: REQUEST CHANGES (CRITICAL 1건 해소 전 배포 불가)
> CRITICAL부터 고치자. 수정안대로 진행해줘

```

이 CRITICAL은 사람 리뷰에서 두 번이나 통과됐던 코드였습니다. "리뷰어 에이전트는 지치지 않고, 금요일 오후에도 꼼꼼하다" — 지우의 회고입니다.

6장. 훅(Hooks) — 사람이 안 지켜봐도 되는 가드레일



훅은 Claude Code의 특정 이벤트(도구 실행 전/후, 세션 시작, 프롬프트 제출 등)에 자동으로 실행되는 셸 스크립트입니다. "Claude가 알아서 잘하겠지"를 "구조적으로 사고가 불가능하다"로 바꾸는 장치입니다.

실행 절차

1. **이벤트 선택**: 언제 개입할지 정합니다.

이벤트	시점	대표 용도
PreToolUse	도구 실행 직전	위험 명령 차단
PostToolUse	도구 실행 직후	결과 검사, 시크릿 필터링
UserPromptSubmit	프롬프트 제출 시	컨텍스트 자동 주입
SessionStart	세션 시작 시	상태 복원, 리마인더
Stop	응답 완료 시	작업 로그 기록

2. **스크립트 작성**: 예 — 파괴적 DB 명령을 차단하는 가드.

```
#!/bin/bash
# db-guard.sh (PreToolUse, matcher: Bash)
input=$(cat)
cmd=$(echo "$input" | jq -r '.tool_input.command')

if echo "$cmd" | grep -qiE "(drop table|truncate|prisma migrate reset)"; then
  echo '{"decision": "block",
    "reason": "파괴적 DB 명령은 수동 실행 필요"}'
  exit 0
fi
exit 0
```

3. **등록**: settings.json의 `hooks` 섹션에 연결합니다.

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{ "type": "command",
                  "command": "~/.claude/hooks/db-guard.sh" }]
    }]
  }
}
```

4. **테스트**: 차단 대상 명령을 일부러 요청해 혹이 막는지 확인합니다.

설정팩 혹은 핵심 3종

- `output-secret-filter.sh` (PostToolUse): 도구 출력에서 API 키·토큰 패턴을 감지해 마스킹 — 시크릿이 대화 로그에 남는 사고 방지
- `db-guard.sh`: DROP/TRUNCATE 등 파괴적 DB 명령 차단
- `rate-limiter.sh` (mcp__* matcher): MCP 도구 호출 빈도 제한 — 외부 API 요금 폭탄 방지

혹과 권한(3장)의 차이: 권한은 "무엇을 할 수 있나"의 정적 목록, 혹은 명령 내용을 스크립트로 검사하는 동적 로직입니다. 정규식·조건문이 필요한 규칙은 혹으로, 단순 목록은 권한으로.

CASE STUDY 현장 적용 — 개발 DB가 살아남은 오후

하람이 시드 데이터를 갈아엎으려다 생긴 일입니다. 요청 자체는 평범했지만, 그 안에 파괴적 명령이 숨어 있었습니다.

```
claude - 세션 기록

> 시드 데이터가 꼬였어. 주문 테이블 비우고 다시 시드 넣어줘
• Bash(npx prisma migrate reset --force)
  └─ 🛑 Hook(db-guard): BLOCKED
    "파괴적 DB 명령은 수동 실행 필요"
• 혹 정책에 따라 reset은 실행할 수 없습니다. 대안:
  1. 주문 테이블만 지우는 스크립트를 만들어 검토 후 직접 실행
  2. 개발 DB를 새 스키마로 복제해 시드 (원본 보존)
  어느 쪽으로 할까요?
> 2번으로. 원본은 남겨두자
```

`migrate reset` 은 주문 테이블만이 아니라 **DB 전체를 초기화**하는 명령입니다. 그날 개발 DB에는 QA가 사흘간 쌓은 재현 데이터가 들어 있었습니다. 혹 스크립트 14줄이 지킨 것은 데이터가 아니라 사흘이었습니다.

7장. MCP 서버 — Claude의 손을 외부 세계로 확장한다



MCP(Model Context Protocol)는 Claude Code에 외부 도구를 연결하는 표준입니다. DB 조회, 라이브러리 최신 문서, 웹 검색, GitHub 조작을 대화 안에서 직접 수행합니다.

실행 절차

1. **서버 선택**: 필요한 것만. MCP 서버 하나하나가 컨텍스트와 시작 시간을 잡아먹으므로 "일단 다 설치"는 하수입니다.
2. **등록**: CLI로 등록하거나 `.mcp.json` 을 프로젝트에 커밋합니다.

```
claude mcp add context7 -- npx -y @upstash/context7-mcp@latest
```

3. **인증**: 토큰은 설정 파일에 하드코딩하지 말고 `${GITHUB_TOKEN}` 참조로.
4. **활용**: 등록 후에는 자연어로 요청하면 됩니다.

검증된 범용 조합 (설정팩 기본 구성)

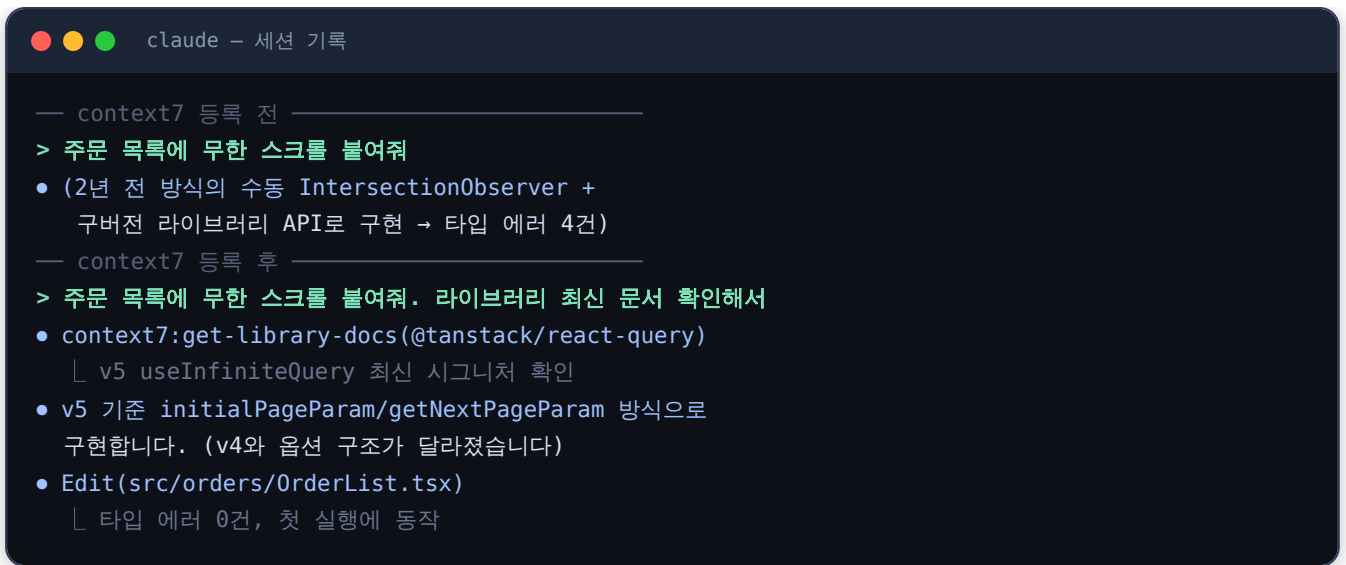
서버	용도	체감 효과
context7	라이브러리 최신 문서 주입	구버전 API로 코드 짜는 문제 해결
memory	세션 간 지식 그래프	"지난번에 말했잖아"가 통하게 됨
github	PR/이슈/리포 조작	리뷰→수정→PR 사이클을 대화로
exa	AI 웹 검색	에러 메시지 최신 해결책 검색

비용 주의: MCP 도구가 외부 유료 API를 쓰는 경우가 있습니다. 설정팩의

`expensive-mcp-warning.sh` 혹처럼 고비용 호출 전 경고 장치를 권장합니다.

CASE STUDY 현장 적용 — 구버전 API 지옥에서 탈출

하람의 고민: Claude가 React 코드를 짤 때 학습 시점의 낡은 패턴을 쓰는 것. context7 등록 전후가 이렇게 같았습니다.



"최신 문서 확인해서"라는 한 마디와 MCP 서버 하나가, 시행착오 왕복을 첫 실행 성공으로 바꿨습니다.

8장. 실전 워크플로우 4종 — 조합이 실력이다

WORKFLOW 1	WORKFLOW 2	WORKFLOW 3	WORKFLOW 4
새 기능 개발	버그 수정	보안 감사	병렬 리팩토링
/plan → /tdd → /code-review	재현 테스트 → 최소 수정 → /verify-loop	/security-review → 수정 → 재검사	/worktree-start → 리뷰 → 머지

지금까지의 재료(커맨드·에이전트·훅)를 실무 시나리오로 조합합니다. 이 장이 이 책의 본론입니다.

워크플로우 1: 새 기능 개발 (TDD)

/plan 알림 설정 기능 (계획 검토·승인)	← planner가 구현 계획 수립
/tdd	← 테스트 먼저 작성 강제
/code-review	← 심각도 기반 리뷰
/quick-commit	← 컨벤션 커밋

워크플로우 2: 버그 수정

1. 버그 재현 테스트를 먼저 작성하게 한다
2. 원인 분석 — 수정 전에 가설을 설명하게 한다
3. 최소 수정으로 테스트 통과
4. /verify-loop 로 전체 테스트·린트·빌드 확인

워크플로우 3: 보안 감사 — /security-review → 항목별 수정 → 재검사 반복

워크플로우 4: 대규모 리팩토링(병렬) — /worktree-start 로 격리 브랜치에서 진행, 메인 작업과 동시 수행 후 리뷰·머지

CASE STUDY 현장 적용 — 알림 기능, 계획부터 머지까지 한 사이클

리프오더의 "재고 부족 알림" 기능을 워크플로우 1로 처리한 실제 흐름입니다.

```
claude - 세션 기록

> /plan 로스터리별 재고 부족 알림 (기준치 이하 시 이메일)
• Task(planner)
  └─ 계획: ① 스키마(alert_rules) ② 재고 체크 잡
        ③ 알림 발송 모듈 ④ 설정 API - 4단계, 예상 파일 9개
        리스크: 중복 발송 방지(멥등성) 설계 필요
> 승인. /tdd 로 1단계부터
• Write(src/alerts/rules.test.ts) ← 테스트 먼저 (RED)
  └─ 7 tests, 7 failed ✓(의도된 실패)
• Write(src/alerts/rules.ts) ← 최소 구현 (GREEN)
  └─ 7 passed
• 리팩토링: 중복 발송 방지 로직을 idempotencyKey로 분리
  └─ 7 passed (유지)
> /code-review
• Task(code-reviewer)
  └─ [MEDIUM] 1건, [LOW] 2건 - 수정 후 APPROVE
> /quick-commit
• [feat/stock-alerts 4d2e91a] feat(alerts): 재고 부족 알림 규칙 엔진
```

지우의 기록에 따르면 유사 규모 기능의 이전 평균 소요는 이를 반, 이 사이클로는 하루가 채 걸리지 않았습니다. 속도 보다 중요한 변화는 모든 기능에 테스트와 리뷰 기록이 남기 시작했다는 것입니다.

기능 1건 소요

2.5일 → 1일 미만

계획-TDD-리뷰 사이클

테스트 커버리지

58% → 81%

/tdd 강제 후 6주

리뷰 기록

전무 → 전 기능

에이전트 리포트 저장

9장. 컨텍스트와 비용 관리 — 오래 달리기 위한 기술

실무에서 부딪히는 두 벽: 컨텍스트 한계와 토큰 비용.

컨텍스트 관리 원칙

1. **한 세션 = 한 작업**: 작업이 끝나면 `/clear`. 여러 작업을 한 세션에 섞으면 컨텍스트가 오염되어 품질이 떨어집니다.
2. `/compact` 는 마일스톤에서: 자동 압축이 뜨기 전에 능동적으로.
3. **탐색은 서브에이전트로**: 수백 파일을 읽은 흔적이 메인 컨텍스트에 남지 않습니다.
4. **CLAUDE.md 다이어트**: 모든 세션에 로드됩니다. 비대하면 매 세션이 무겁습니다.

비용 절약 전략

- 기계적 작업(포매팅, 단순 변환, 문서 갱신)은 하위 모델로. 에이전트 정의의 `model` 필드로 역할별 모델을 고정하세요.
- 같은 질문을 반복하지 말고 CLAUDE.md나 memory MCP에 기록하세요. "기억 비용"은 한 번, "재설명 비용"은 매번입니다.
- 대형 작업 전 `/plan`으로 범위를 좁히는 것 자체가 최대의 비용 절감입니다.

CASE STUDY 현장 적용 — 월말 청구서를 줄인 세 가지 조치

리프오더 팀이 첫 달 사용량 리포트를 보고 취한 조치입니다.

```

claude - 세션 기록

> /cost
• 이번 세션: $4.31
  이번 주 상위 비용 세션:
  1. $18.20 - "전체 코드베이스 분석 + 리팩토링 + 버그 3건"
    (한 세션에 작업 5개 혼재, 컨텍스트 3회 자동압축)
  2. $9.75 - doc 갱신 37회 (상위 모델로 수행)
  
```

조치: ① 작업마다 `/clear` 습관화 ② `doc-updater` 에이전트의 `model` 을 하위 모델로 변경 ③ 탐색성 질문은 Explore 에이전트로 위임. 다음 달 같은 작업량 기준 청구액이 약 40% 내려갔습니다. 비용 최적화의 대부분은 모델 선택이 아니라 **세션 위생**에서 나옵니다.

10장. 트러블슈팅 FAQ — 자주 부딪히는 문제들

Q. Claude가 CLAUDE.md 규칙을 무시해요. 규칙이 너무 많거나 모호할 가능성이 높습니다. 명령형 한 문장으로 줄이고, 중요한 것을 상단에. 반복되면 그 규칙을 흑으로 승격하세요 — 규칙은 어길 수 있지만 흑은 못 어깁니다.

Q. 매번 권한 확인 버튼 누르는 게 지쳐요. 3장의 `permissions.allow` 에 안전한 명령을 추가하세요. 단 `Bash(*)` 전체 허용은 금물.

Q. 긴 작업을 시키면 중간에 방향이 틀어져요. 시작 전 `/plan`으로 계획을 문서화하고, 중간마다 "계획 대비 점검"을 요청하세요. 설정팩의 `/checkpoint`가 이 용도입니다.

Q. 팀원과 설정을 공유하려면? `.claude/` (commands, agents, settings.json)와 `CLAUDE.md`, `.mcp.json`을 git에 커밋. 개인 API 키·취향은 `settings.local.json`으로 분리.

Q. Claude가 위험한 명령을 실행할까 무서워요. 기본값은 모든 수정·실행에 승인이 필요해 안전합니다. 더 조이라면 deny 권한(3장) + 가드 흑(6장) + git 브랜치 보호를 조합하세요. 이 3중 구조면 자율 모드로 돌려도 사고 범위가 브랜치 하나로 제한됩니다.

Q. 설정팩은 어떻게 설치하나요? 부록 참고. 설치 스크립트가 `~/.claude`에 심볼릭 링크를 만들어 모든 프로젝트에서 커맨드·에이전트를 공유합니다.

부록. 동봉 설정팩(claude-forge) 설치 가이드

이 책의 모든 예제가 들어 있는 설정팩은 오픈소스(MIT)로 제공됩니다.

설치 (macOS / Linux / WSL)

```
git clone <설정팩 저장소 URL>
cd claude-forge
./install.sh
```

설치 후 홈 디렉터리 구조는 다음과 같습니다.

디렉터리 구조

```
~/ .claude/
├─ CLAUDE.md           ← 전역 규칙 (rules/를 참조)
├─ settings.json       ← 권한·혹 설정
├─ commands/          ← 슬래시 커맨드 35종
│   └─ plan.md
│   └─ tdd.md
│   └─ code-review.md
│   └─ ...
├─ agents/             ← 서브에이전트 11종
│   └─ code-reviewer.md
│   └─ planner.md
│   └─ ...
├─ hooks/              ← 자동화 혹 15종
│   └─ db-guard.sh
│   └─ output-secret-filter.sh
│   └─ ...
├─ rules/              ← 규칙 파일 8종
└─ skills/             ← 스킬 15종
```

커스터마이징 원칙

- 원본을 직접 수정하지 말고 `-custom` 접미사 폴더에 복사 후 수정 (업데이트 시 충돌 방지)
- 프로젝트 고유 설정은 프로젝트 `.claude/` 에, 범용 설정은 전역에
- 안 쓰는 커맨드는 과감히 삭제 — 커맨드 목록도 컨텍스트를 차지합니다

맺으며 — 도구가 아니라 시스템을 만드세요

Claude Code를 잘 쓰는 사람과 못 쓰는 사람의 차이는 프롬프트 실력이 아닙니다. 반복되는 것을 시스템(커맨드·에이전트·혹)으로 승격시키는 습관의 차이입니다.

- 같은 지시를 두 번 타이핑했다면 → 커맨드로
- 같은 역할을 두 번 시켰다면 → 에이전트로
- 같은 실수를 두 번 봤다면 → 흑으로

리프오더 팀이 이 책의 순서 그대로 6주를 보낸 뒤 남은 것은 빨라진 개발 속도만이 아니라, **사람이 지켜보지 않아도 안전한 개발 환경**이었습니다. 이 책의 설정팩은 그 시스템의 출발점입니다. 그대로 쓰기보다, 여러분의 프로젝트에서 반복되는 것들을 하나씩 엮어가며 **여러분만의 포지(대장간)**로 키워가시길 바랍니다.